Experiment 5: Design and Implementation of Pipelined FIR Filter

Aslin Garvasis
EE24BTECH11008

1 Objective

To design and implement a pipelined FIR filter using Verilog, analyze hardware resource utilization using Quartus, and verify the output using MATLAB.

2 FIR Filter Theory

The FIR filter output is given by:
$$[\ y[n] = \sum_{k=0}^{N-1} h[k] \cdot x[n-k]]$$

3 Design Methodology

The experiment was carried out in the following steps:

1. FIR filter was designed using Verilog for both pipelined and non-pipelined architectures.
2. The designs were synthesized in Quartus Prime Lite to obtain resource utilization.
3. Testbenches were created to apply input signals and generate output data in text files.
4. The output data from Verilog simulation was processed using MATLAB.
5. MATLAB was used to plot the output signals for analysis.

4 Pipelined FIR Architectures

Three pipelined FIR architectures were implemented:

- Direct Form Pipeline
- Optimized Form Pipeline
- Genvar-Based Pipeline

Pipeline registers were inserted between stages to improve performance and reduce critical path delay.

5 Simulation and Output Generation

Testbenches were used to provide input signals (950 Hz, 1100 Hz, and 2000 Hz) to the FIR filter.

The simulation was carried out using ModelSim, and the output values were written into text files.

6 MATLAB Plotting

The output text files generated from Verilog simulation were imported into MATLAB.

The data was converted from fixed-point format to floating-point representation, and output waveforms were plotted.

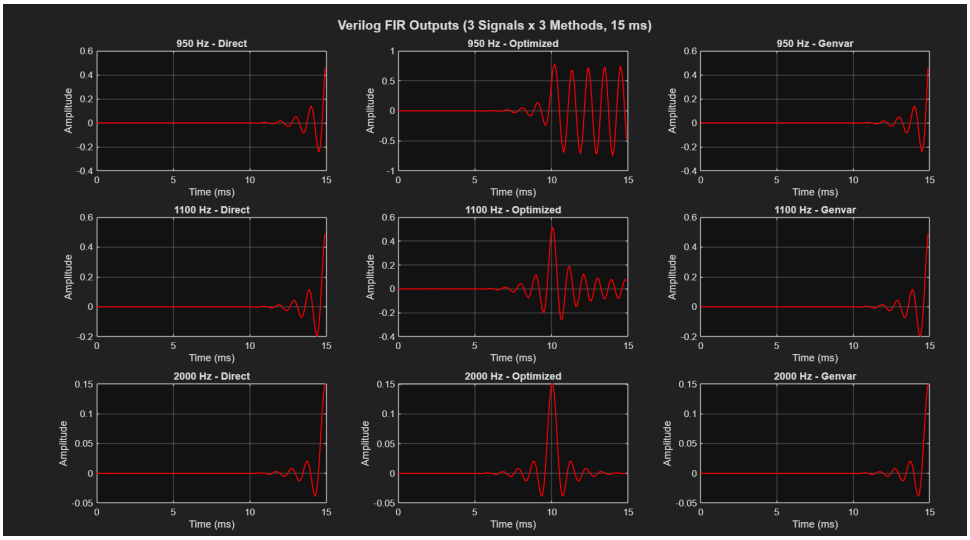

Figure 1: Output waveform obtained from MATLAB

7 Resource Utilization

The synthesized results obtained from Quartus are shown below.

7.1 Non-Pipelined Designs

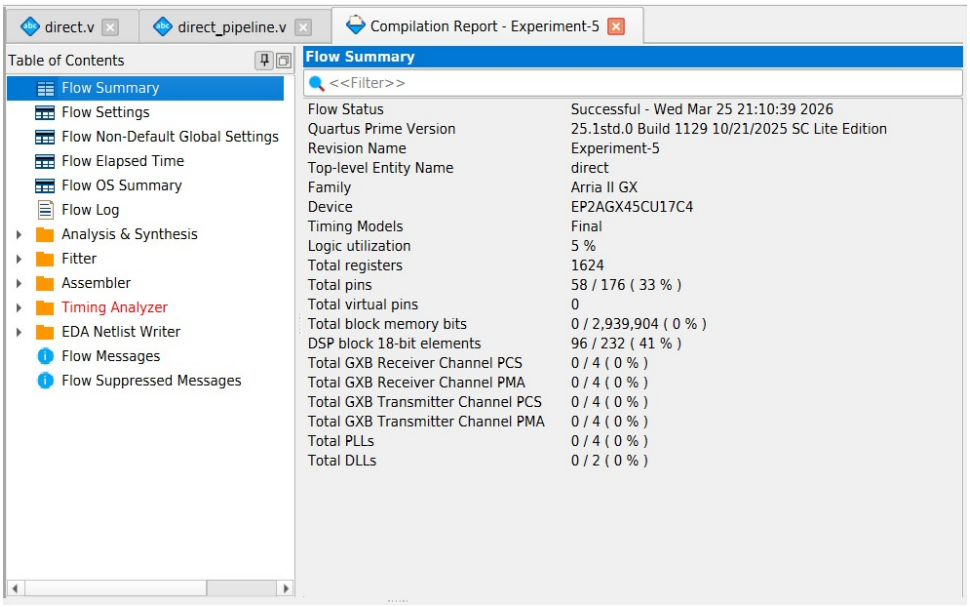

Figure 2: Direct Form (Non-Pipeline)

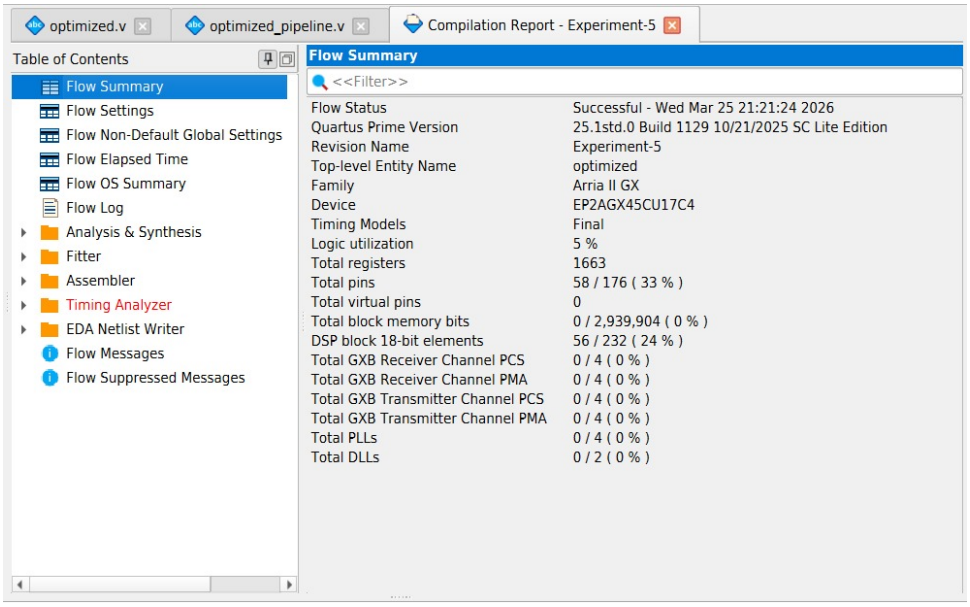

Figure 3: Optimized Form (Non-Pipeline)

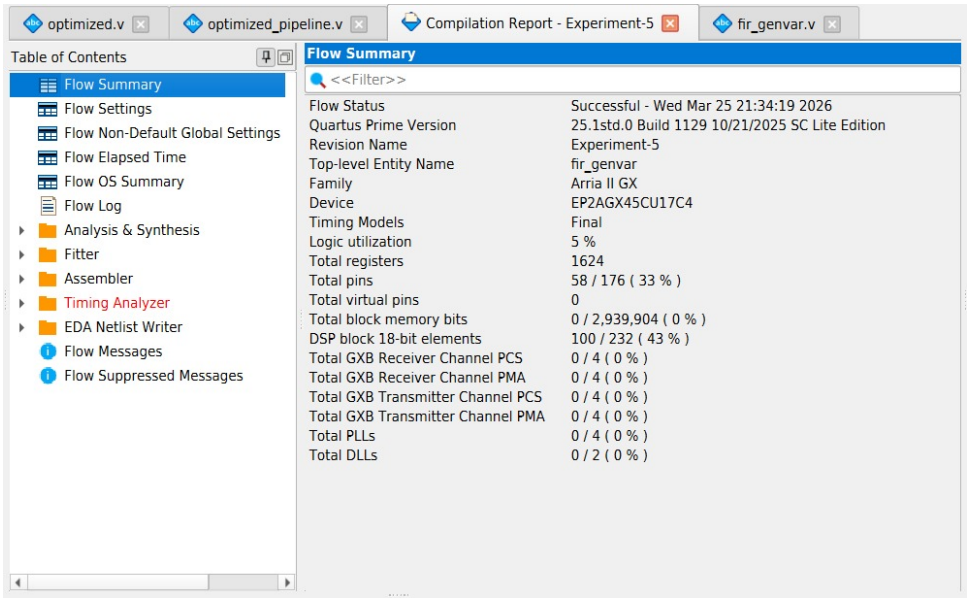

Figure 4: Genvar Form (Non-Pipeline)

7.2 Pipelined Designs

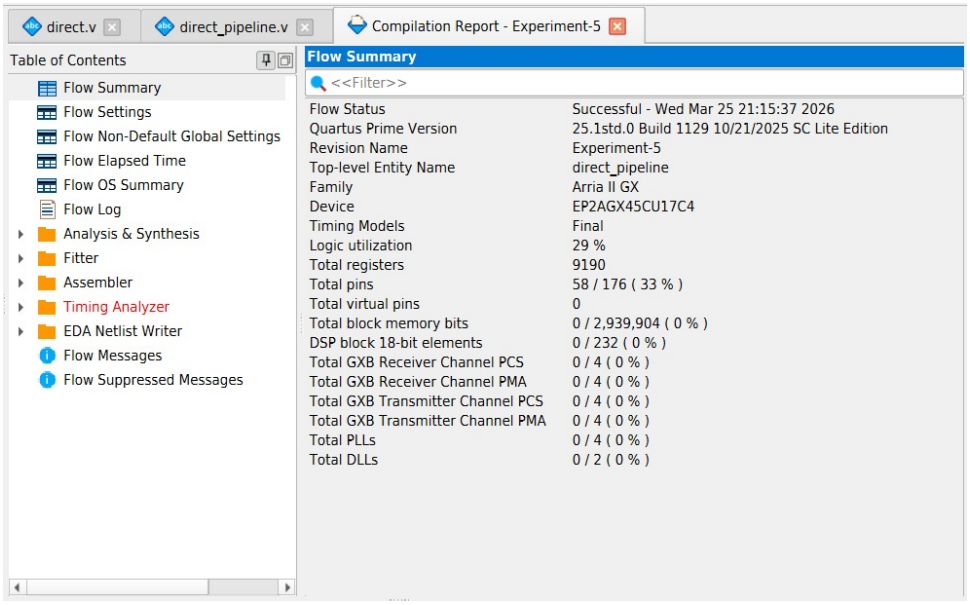

Figure 5: Direct Form Pipeline

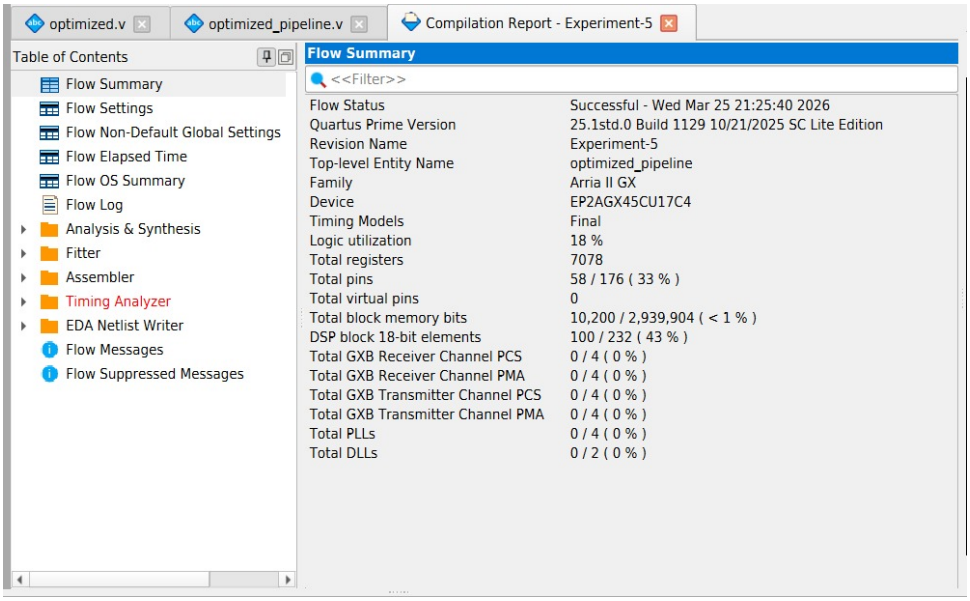

Figure 6: Optimized Pipeline

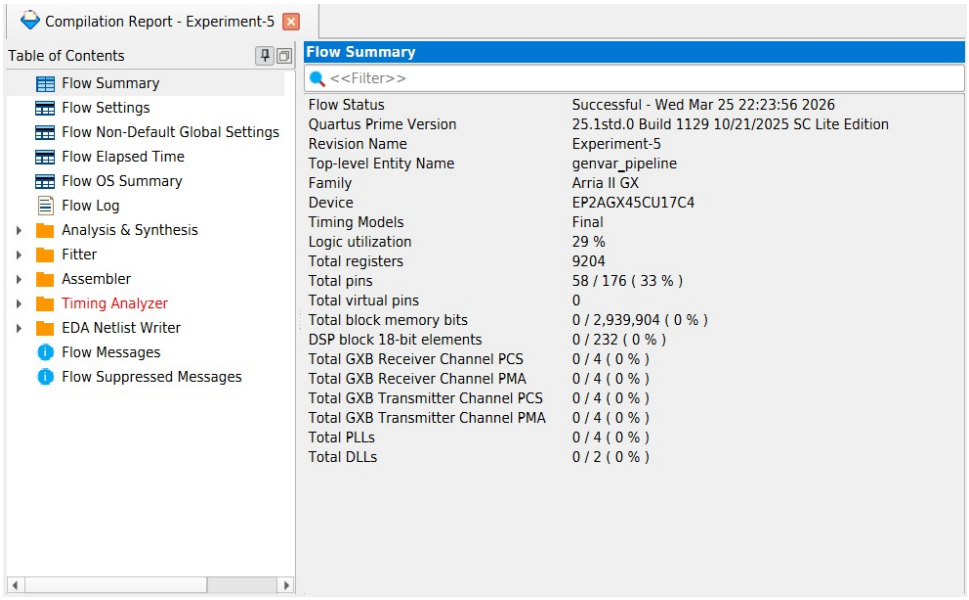

Figure 7: Genvar Pipeline

8 Analysis of Results and Performance

9 Analysis of Results and Performance

The FIR filter designs were synthesized using Intel Quartus Prime Lite, and the hardware resource utilization was analyzed for both pipelined and non-pipelined architectures.

From the obtained results, it is observed that pipelined designs consume more hardware resources compared to non-pipelined designs.

- **Registers:** The number of registers increases significantly in pipelined designs. For example, the optimized pipelined design utilizes approximately **7078 registers**,

whereas the non-pipelined design uses considerably fewer registers (typically in the range of a few thousand). This increase is due to the insertion of pipeline registers between computation stages.

- **Logic Elements (ALMs):** Logic utilization also increases slightly in pipelined designs due to additional control logic and intermediate storage.
- **Memory Usage:** Memory usage remains relatively low in both cases (less than 1
- **DSP Blocks:** The number of DSP blocks remains approximately the same (e.g., around **100 DSP blocks** for a 100-tap filter), since the number of multiplications does not change between pipelined and non-pipelined implementations.

9.1 Timing Performance Analysis

One of the most significant differences between pipelined and non-pipelined FIR filters is the timing performance.

- **Non-Pipelined Design:** All multiplications and additions are performed within a single clock cycle. This results in a **long critical path delay**, as the signal must propagate through multiple multipliers and adders sequentially. Hence, the maximum operating frequency (f_{max}) is low.
- **Pipelined Design:** The computation is divided into multiple stages by inserting registers. Each stage performs only a part of the computation. This significantly **reduces the critical path delay**.

Observation:

- Non-pipelined delay $\approx$ delay of (multiplier + adder chain)
- Pipelined delay $\approx$ delay of (single stage, e.g., one multiplier or adder)

Thus, pipelining allows a much higher clock frequency, improving system speed.

9.2 Pipeline Efficiency

Pipeline architecture improves the performance of the FIR filter by increasing throughput.

- Once the pipeline is filled, the system produces one output per clock cycle.
- This results in high data processing speed.
- The overall efficiency improves for continuous data streams.

However, this improvement comes at the cost of increased hardware usage.

9.3 Working of Pipeline Architecture

In a pipelined FIR filter, the computation is divided into multiple stages:

1. Input samples are stored in a shift register (delay line).
2. Each stage performs multiplication with filter coefficients.
3. Partial results are passed through pipeline registers.
4. Final output is obtained after multiple clock cycles.

Each pipeline stage operates simultaneously on different data samples, similar to an assembly line, which increases parallelism.

9.4 Latency and Throughput Comparison

- **Latency:** Latency is the number of clock cycles required to produce the first output.
  - Non-pipelined FIR: Low latency (output generated in 1 clock cycle)
  - Pipelined FIR: High latency (approximately equal to number of pipeline stages, e.g., $\sim$100 cycles for a 100-tap filter)
- **Throughput:** Throughput is the rate at which outputs are produced.
  - Non-pipelined FIR: Lower throughput due to longer clock period
  - Pipelined FIR: High throughput (one output per clock cycle after pipeline is filled)

9.5 Memory vs Register Trade-off

Although both designs use minimal block memory, there is a significant difference in storage style:

- Non-pipelined design relies on fewer registers and performs operations sequentially.
- Pipelined design introduces a large number of registers (e.g., thousands of flip-flops), effectively increasing storage at intermediate stages.

Thus, pipelining increases **register-based memory usage** rather than block RAM usage.

9.6 Quartus Design Flow

The following steps were followed in Quartus Prime Lite:

1. Verilog design files were created for FIR filter architectures.
2. The project was compiled using Quartus to perform synthesis and fitting.
3. Resource utilization was obtained from the compilation report.
4. The design was analyzed for logic usage, registers, and DSP blocks.
5. Simulation was performed using ModelSim to verify functionality.

9.7 Overall Performance Trade-off

- Pipelined FIR filters achieve:
  - Higher operating frequency
  - Reduced critical path delay
  - Higher throughput
- However, they require:
  - More registers (e.g., ∼7000+)
  - Slightly higher logic utilization
- Non-pipelined FIR filters:
  - Use fewer hardware resources
  - Have lower latency
  - Operate at lower speed

10 Observations

- Pipelined designs use more registers due to additional pipeline stages.
- Logic utilization increases in pipelined implementations.
- Pipeline architecture improves system performance by reducing critical path delay.
- Output waveforms obtained from MATLAB confirm correct filter operation.

11 Conclusion

The pipelined FIR filter was successfully implemented using Verilog.

Resource utilization was analyzed using Quartus, and output results were verified using MATLAB. The pipeline architecture improved performance at the cost of increased hardware usage.